DL_T11_3C: Unsupervised Anomaly Detection in Motor-Bearing Vibration (CWRU) Using Autoencoders

Task 3, Team 11, variant 3C · CM701PC / CM751PC Deep Learning, MGIT (Autonomous) · Code: [FILL: GitHub link or ZIP name]

Roll no.	Role (department allotment)	Contribution (to be confirmed by the team)
23261A6649	Data lead	Downloaded and inspected CWRU files; time-ordered splits, windowing, scaler; report §2
23261A6644	Model lead	Built and trained Dense and CNN autoencoders (7 sizes, 3 seeds); README, reproducible code; report §3
23261A6622	Evaluation and report lead	Metrics, thresholds A and B, cost analysis, plots, target checks; answers to Q1 to Q3; compiled report

1. Problem understanding

A plant records vibration from motor bearings. Breakdowns are rare, almost all data is normal, and few faults are labelled, so we built an unsupervised warning system: it learns normal vibration and flags departures from it. Targets: recall on fault events ≥ 0.85, fewer than 2 false alarms per 24 h, detection delay under 5 min (or within 3 readings for data sampled slower than once a minute), and no labelled fault data in training. A lower threshold raises recall and false alarms together, and engineers will switch the system off if false alarms are frequent. A missed fault costs about Rs 4,00,000 and a false alarm about Rs 2,000 (200 to 1). Bottleneck size is a second trade-off: too wide risks copying the input, too narrow underfits.

Important caveat. CWRU [1] is a set of short laboratory recordings with seeded faults, not a continuous plant stream. We have 141.4 s of normal data in total and only 27.7 s of normal test data, so false alarms per 24 h are extrapolated and cannot be statistically verified. This benchmark is also very easy (§5): even a plain signal-energy threshold nearly separates normal from faulty windows. Our results describe this benchmark and are likely optimistic for a real plant.

2. Data preparation and exploratory analysis

We used the 12 kHz drive-end data: 4 normal files (97 to 100; load 0 to 3 HP; 20.3 s, 40.3 s, 40.3 s, 40.5 s) and 36 fault files (inner race IR, ball B, outer race OR at 6 o'clock; 0.007, 0.014, 0.021 in; load 0 to 3 HP; about 10.2 s each). No file has missing values. Normal files have RMS 0.066 to 0.074 and kurtosis near 0 (−0.24 to −0.04). Fault files differ strongly: mean RMS 0.137 (B), 0.317 (IR), 0.423 (OR) and mean kurtosis 4.3 (B), 8.4 (IR), 8.1 (OR), consistent with impact-type defect signals. Signals were cut into windows of 1024 samples (85 ms).

Each normal file was split in time: first 60% train, next 20% validation, last 20% test, with a one-window gap. Training windows overlap 50%; validation and test windows do not. The scaler (mean 0.0122, std 0.0661) was fitted on training data only. Fault files were never used for training: faults at loads 0 and 1 tuned the threshold; faults at loads 2 and 3 were test only.

Split	Windows	Fault files
Train (normal only)	1,984	0
Validation normal	325	0
Validation fault (load 0, 1 HP)	2,134	18
Test normal	325	0
Test fault (load 2, 3 HP)	2,136	18

3. Models and experiments

Both models rebuild a 1024-sample window and are trained on normal windows only; the anomaly score is the mean squared reconstruction error (in units of the scaled signal) [3], [4]. **Dense AE (baseline):** hidden layers 512 and 256 (or the bottleneck size if larger), linear code layer, mirrored decoder. **1D CNN AE (improved):** four stride-2 Conv1D layers (16, 32, 64, 64 filters, kernel 7) compress to 64 steps, a Conv1D code layer of C channels (latent = 64 × C), then four transposed convolutions. Adam (1e-3) [6], batch 128, at most 60 epochs, early stopping on normal validation loss, 3 seeds per configuration, TensorFlow [7]. Test data were never used to choose a model.

Table 1. Model and bottleneck comparison (3 seeds; mean, ± sd where non-zero)

Model	Latent	Params	Val AP	Test AUC	Fault/ normal error ratio	Train MSE	s / 10k windows
Dense	1024	6,297,600	1.000	1.000 ± 0.000	23.9 ± 0.1	0.151	9.8
Dense	256	1,444,608	1.000	1.000 ± 0.000	27.7 ± 0.7	0.154	7.4
Dense	64	1,346,112	1.000	1.000 ± 0.000	25.7 ± 0.9	0.183	7.0
Dense	16	1,321,488	1.000	1.000 ± 0.000	18.9 ± 0.0	0.323	7.0
CNN	1024	104,017	1.000	1.000 ± 0.000	995 ± 81	0.0068	11.8
CNN	256	96,325	1.000	1.000 ± 0.000	245 ± 2.7	0.030	13.0
CNN	64	94,402	1.000	1.000 ± 0.000	27.7 ± 0.2	0.275	11.0

Timings are for scoring windows on the CPU of our run environment and will differ on Colab. **Every configuration has validation AP = 1.000, so the notebook's tie-break (smaller latent size) selected Dense latent 16 and CNN latent 64. The selection is not evidence that these sizes are better.**

Thresholds were chosen on validation data only. **A** is the best-F1 threshold (middle of the tied range): Dense 0.904, CNN 0.855. **B** minimises Rs 4,00,000 × missed fault events + Rs 2,000 × false alarms per 24 h over thresholds and k = 1, 2, 3 consecutive alarms: Dense 2.014, CNN 2.072, both with k = 1. A fault event counts as missed if no alarm occurs within 3 readings. The cost assumes one 24-hour period in which each test fault occurs once, which is our modelling assumption.

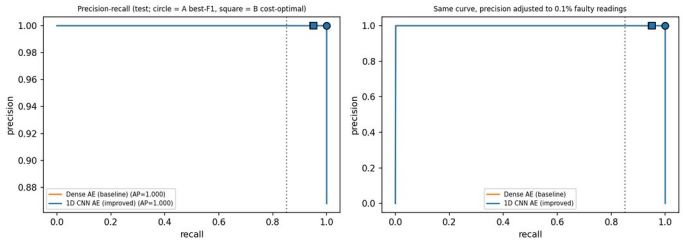
4. Results against every target

Our readings are 85 ms windows, much faster than once a minute, so the 5-minute delay rule applies; we also report the stricter 3-reading rule. Test split: 18 fault files (loads 2 and 3) and 325 normal windows (27.7 s).

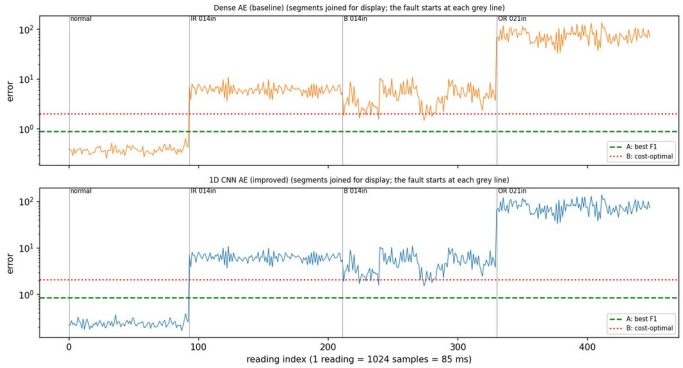
Target	Required	Dense AE, A / B	CNN AE, A / B	Met?
Recall, window level	≥ 0.85	1.000 / 0.949	1.000 / 0.952	Yes
Recall, events within 3 readings	≥ 0.85	18/18 / 18/18	18/18 / 18/18	Yes
False alarms per 24 h	< 2	0 observed in 27.7 s (95% upper bound 11,492)	same	Point estimate met; not statistically verifiable
Detection delay	< 5 min or ≤ 3 readings	mean 1.00 / 1.06 readings (0.085 / 0.090 s); max 1 / 2	same	Yes
Labelled fault data in training	none	none	none	Yes

Window precision was 1.000 at both operating points and the test window false-positive rate was 0. Cost on the test split under our cost model: Rs 0 for A and for B for both models (no missed events, no false alarms), so the cost function cannot separate them. Share of fault windows flagged at B (test): inner race 1.000, ball 0.968 (Dense) / 0.965 (CNN), outer race 0.881 / 0.891; the weakest file

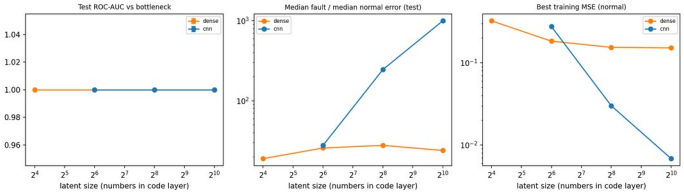
was OR 0.014 in at 3 HP (0.454 Dense, 0.479 CNN). At A every fault window of every type was flagged.



Precision-recall on the test split; circles mark threshold A, squares threshold B.



Reconstruction error against reading index (normal, then three fault files joined for display). Dashed: A; dotted: B.



Bottleneck comparison: test AUC, fault/normal error ratio and training MSE against latent size.

5. Answers to the report questions

Q1 (wide bottleneck). In principle, a code layer as wide as the input can learn close to a copy, so faults are rebuilt as well as normal windows and their error does not rise; a narrow bottleneck forces the model to keep only what is common in normal vibration, and one that is too narrow underfits. **Our data did not show this failure.** Dense with latent 1024 had test AUC 1.000 and a fault/normal error ratio of 23.9, no worse than latent 16 (AUC 1.000, ratio 18.9) or latent 64 (25.7). Two measured facts explain part of it. First, the wide dense model did not learn a copy: its training MSE was 0.151 (a copy would give about 0; scaled signal variance is 1) after early stopping at about 26 epochs on 1,984 training windows. Second, fault windows carry much more energy (RMS 2 to 6 times normal) and both models leave most of it unreconstructed: the fault error curves of the two models almost coincide in the error-stream figure, and a plain signal-energy score reaches test AUC 0.99999 with no model at all. The CNN with latent 1024 did come close to copying normal data (training MSE 0.0068) and still gave the largest separation (ratio 995); we think filters trained only on low-amplitude normal windows do not carry over to high-amplitude impacts, but we did not test this. We therefore cannot claim empirical support for the wide-bottleneck failure on CWRU; showing it would need weaker faults or longer training of the wide dense model, which we did not run.

Q2 (improved vs baseline). Both models detect all 18 test fault events with 0 false positives, and both have AUC and AP of 1.000, so detection performance does not separate them. At equal latent size 64 the CNN has 94,402 parameters against 1,346,112 for the dense model (14 times fewer) and a similar error ratio (27.7 ± 0.2 vs 25.7 ± 0.9). The CNN's ratio is much larger at latent 256 (245 vs 27.7) and 1024 (995 vs 23.9), a wider margin between normal and fault scores, but with no extra detection benefit on this data. The CNN was also slightly slower per window (11.0 vs 7.0 s per 10k windows). A dense layer treats the 1024 samples as unrelated inputs, while convolutions share one

small filter across positions, so a short repeating impact is learned once and found anywhere in the window. The CNN is therefore a more parameter-efficient model with a wider margin, not a more accurate detector here.

Q3 (threshold and cost). A (best F1) was 0.904 (Dense) and 0.855 (CNN); B (cost-optimal) was 2.014 and 2.072, i.e. **higher** than A, the opposite of the usual expectation for a 200 to 1 cost ratio. The data separate perfectly, so many thresholds give zero misses and zero false alarms; the cost is Rs 0 for both A and B, and our rule picks the middle of the zero-cost range, which is wider than the best-F1 range because an event still counts as detected if any of its first 3 windows alarm. B costs window recall (0.949 / 0.952, and only 45% of windows in the weakest outer-race file) without any measured saving. Because a miss is 200 times costlier than a false alarm and we cannot measure the false-alarm rate, we recommend A (the lower threshold: every fault window flagged, no false positive in 325 normal test windows), to be confirmed on the plant's own data in watch-only mode before alarms are enabled. This is a judgement based on the cost ratio, not a result the cost table proves.

6. Limitations and future work

(i) Normal test data last only 27.7 s, so false alarms per 24 h are unverified (95% upper bound 11,492 per day). (ii) CWRU faults are strong and abrupt: faults start at the first window, so measured delay (1 reading) is optimistic, and a signal-energy score alone gives test AUC 0.99999, so this benchmark cannot show the advantage of a learned model. (iii) Validation and test faults share fault types and sizes and differ only in motor load; unseen fault types are not tested. (iv) All models tied on validation AP, so model selection was by tie-break. (v) One sensor, one sampling rate, drive end only; thresholds may drift with load, speed and temperature. With more time we would test on run-to-failure data with long normal periods (for example NASA IMS), use weaker or earlier faults, train the wide dense model longer, set per-load thresholds, and run in watch-only mode on a real machine.

7. References

[1] Case Western Reserve University Bearing Data Center, "Bearing Data Center," [Online]. Available: https://engineering.case.edu/bearingdatacenter (accessed [FILL date]).

[2] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," *Mech. Syst. Signal Process.*, vols. 64-65, pp. 100-131, 2015.

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 14.

[4] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504-507, 2006.

[5] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proc. MLSDA Workshop*, 2014, pp. 4-11.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[7] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX OSDI*, 2016, pp. 265-283.

Appendix: use of AI tools

The notebook, model and evaluation code, README and the first draft of this report were produced with Claude (Anthropic) at the request of the team. Claude also executed the notebook on the real CWRU files in its own environment and the numbers above were read from the files it wrote (`table_*.csv`, `run_settings.json`). The CWRU files used came from a public GitHub mirror (s-whynot/CWRU-dataset) because the CWRU site was not reachable there. [FILL: state what your team did yourselves: re-running the notebook in Colab, checking numbers, interpreting results, editing the text.] Each member must understand the code before the viva.